Failure-Aware Autonomous Continual RL for Object Manipulation

Siddhesh Girase
School Of Interactive Computing
sgirase3@gatech.edu

Abstract

Vision-Language-Action (VLA) models and diffusion policies demonstrate strong performance in robotic manipulation but often remain brittle to out-of-distribution initializations. In this report, a baseline visual diffusion policy is established on the ManiSkill Push-T task to systematically characterize its failure modes. To try and recover performance without catastrophic forgetting, a "Failure-Aware Autonomous Continual RL" pipeline was implemented. It seemed helpful to utilize a Large Language Model (Gemini 2.5 Pro) to visually analyze failure rollouts and generate a dense reward function. Finally, a PPO-based residual policy was trained utilizing the Policy Decorator architecture. The extensive architectural challenges, manual engineering efforts, and diagnostic insights—such as observing what looked like reward hacking and reverse angle calculations—are documented. Ultimately, despite experiencing a catastrophic failure to improve upon the baseline, this report attempts to provide a comprehensive analysis of the limitations of continual robotic learning in targeted edge cases.

1. Task I: Baseline Diffusion Policy

To establish a comparative baseline, a visual diffusion policy was trained on the ManiSkill `PushT-v1` environment using Imitation Learning (IL). The state representation utilized RGB-D camera inputs alongside proprioceptive states to accurately capture spatial interactions.

1.1. Training Process and Compute

The baseline model (comprising 6.07M parameters) was trained for 50,000 iterations using 16 parallel environments, a batch size of 800, and 4 update epochs. Training was executed on a Google Compute Engine GPU (Colab Pro) and required approximately 2 hours and 28 minutes to complete.

A major hurdle encountered early in the project involved cloud-compute volatility. Initial Colab sessions timed out without persisting checkpoints, necessitating a complete manual retraining of the baseline diffusion policy. To avoid losing progress again, it felt safest to implement a robust, automated Google Drive synchronization script to back up the artifacts.

Hardware utilization was closely monitored to prevent CUDA Out-Of-Memory (OOM) errors:

- **Data Loading Phase:** VRAM usage peaked at 18.1 GB / 22.5 GB, with System RAM reaching 17.0 GB.
- **Training Phase:** VRAM usage stabilized at 15.5 GB, while System RAM settled at 16.4 GB.
- **Loss:** The training loss smoothly converged to approximately 0.0275 by iteration 50,000.

Tensorboard Logs and Convergence: The Tensorboard logs seem to illustrate the baseline training dynamics over the 50,000 iterations.

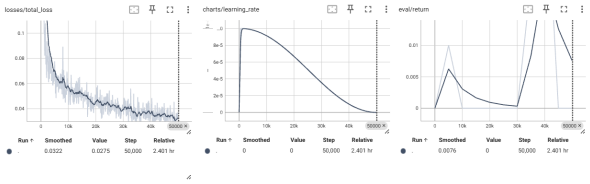

Figure 1. Tensorboard learning curves for the baseline diffusion policy, illustrating what looks like stable loss convergence alongside learning rate schedules and average returns.

1.2. Baseline Evaluation

The frozen base policy was evaluated across 250 rollouts on 5 distinct seeds. Performance was highly constrained:

- **Seed 10:** 0.80%
- **Seed 42:** 0.40%
- **Seed 123:** 0.00%
- **Seed 456:** 1.60%
- **Seed 999:** 0.40%

This resulted in a severely limited average baseline success rate of **0.64%**.

Challenges Faced: A significant challenge during this stage was VRAM management. Loading the high-fidelity RGB-D trajectory HDF5 files required massive memory overhead, necessitating strict batch size adjustments and

headless Vulkan/EGL rendering configurations to successfully complete the backward passes.

2. Task II: Failure Characterization

Qualitative analysis of the baseline's rollouts revealed severe sensitivity to the initial orientation of the T-block relative to the goal state. These were categorized into three distinct failure modes:

- **Mode 1 (Inverted / Severe Misalignment):** Initial block orientation differs from the target by $> 90°$. The robot struggles entirely, often pushing the block completely out of bounds.
- **Mode 2 (Execution / Over-correction):** Initial block orientation is relatively aligned ($\leq 90°$), but the robot overcorrects near the goal, failing to settle the block within the required positional tolerance.
- **Mode 3 (Decoupled Translation/Rotation):** A prominent behavioral collapse where the policy exclusively attempts to translate the block to the goal center without correcting orientation, or solely rotates the block while ignoring global translation.

2.1. Quantitative Characterization

To rigorously evaluate these modes, 1,250 rollouts were swept, dynamically calculating the quaternion yaw difference at step zero.

- **Mode 1 Occurrences:** 598 cases (Success Rate: 0.17%)
- **Mode 2 Occurrences:** 652 cases (Success Rate: 0.46%)

Diagnostic Insights & Challenges: By visually inspecting the generated evaluation footage (specifically `0.mp4` and `3.mp4`), it became apparent that the programmatic failure mode angles were occasionally being calculated in reverse relative to the visual state. This likely happened due to SAPIEN quaternion target wrap-arounds, and this discrepancy required manual alignment of the orientation-tracking scripts to ensure environments were accurately bucketed. Furthermore, ensuring mathematical accuracy when dynamically extracting and comparing quaternion targets ($2 \times \arctan 2(q_z, q_w)$) across batched environments was non-trivial.

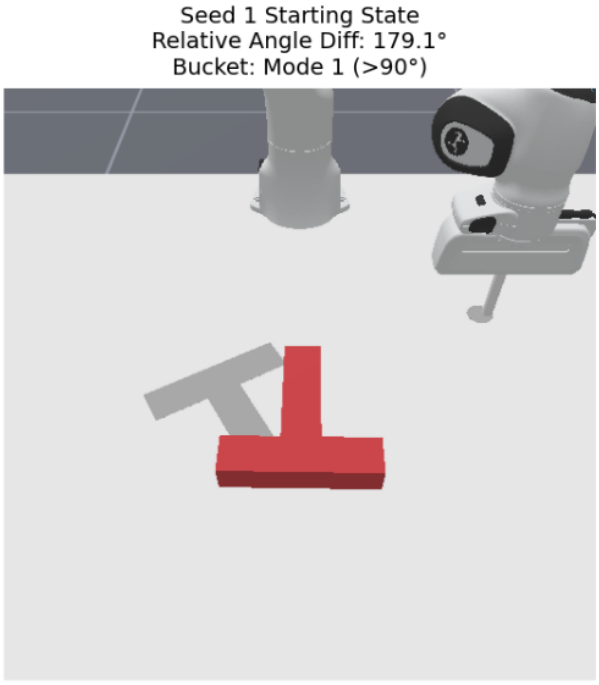

Figure 2. Mode 1: Inverted starting state leading to out-of-bounds failure.

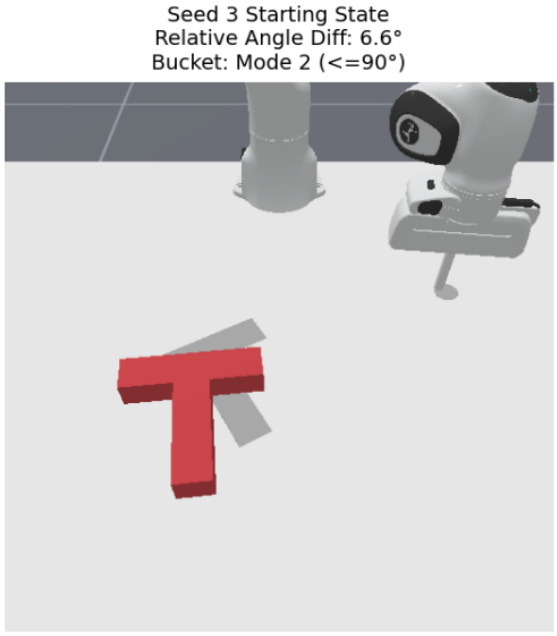

Figure 3. Modes 2 & 3: Execution over-corrections and decoupled translation/rotation behaviors.

3. Task III: LLM-Driven Reward Design

3.1. Literature Review

Recent literature increasingly explores the integration of Large Language Models (LLMs) into the Reinforcement Learning pipeline as automated reward designers and curriculum generators [1, 2]. Frameworks like *Eureka* demonstrate that LLMs can zero-shot generate executable dense reward functions directly from environment source code, iteratively refining them via evolutionary algorithms and fitness feedback [1]. Similarly, systems like *Rosetta* bridge the gap between high-level human intent and low-level robotic control by translating natural language into precisely shaped reward landscapes [2].

The traditional approach to overcoming the limitations

of Imitation Learning has been online RL fine-tuning. However, direct fine-tuning of models like Diffusion Policies is notoriously unstable due to their receding horizon control and complex gradient pathways. The *Policy Decorator* method appears to mitigate this by freezing the base policy and learning a lightweight residual actor [3]. Yet, this introduces a new challenge: defining an appropriate reward signal for the residual policy. Sparse rewards lead to catastrophic sample inefficiency, while dense rewards require meticulous, unscalable human engineering. Here, multimodal LLMs might offer a paradigm shift. Models like Gemini possess extensive spatial priors and code-generation capabilities. By visually analyzing failure trajectories, LLMs act as automated domain experts, hypothesizing dense reward structures that penalize specific geometric failure modes [1]. This intersection of visual foundation models for error diagnosis, LLMs for reward generation, and residual RL for safe policy updates likely forms the frontier of autonomous continuous learning in robotics.

3.2. Prompting and Reward Hacking

Gemini 2.5 Pro was provided with MP4 rollouts of the failures and requested to output a Python reward function mapping SAPIEN pose errors to a scalar reward. A significant amount of time was spent iterating on LLM prompts to resolve what looked like catastrophic reward hacking.

By monitoring the intermediate epoch videos and live Tensorboard logs during the failure loops, the robot was observed actively hovering near the T-block rather than solving the task—which appeared to be a classic exploitation of the LLM's initially proposed exponentially decaying positive reward: $r = \exp(-d \cdot 5.0) \times \exp(-\theta \cdot 2.0)$.

This likely happened because the episodes terminate early upon successful task completion, so yielding a strictly positive reward created a perverse incentive. A lot of manual effort went into overriding this behavior, with the hope that engineering a prompt that strictly enforced a continuous negative distance/angular penalty ($r = -(d \times 10.0) - (\theta \times 2.0)$) alongside modifying `custom_reward.py` to integrate a hardcoded $+50.0$ episodic success bonus would fix it. Furthermore, it seemed like a good idea to implement an episode-configuration sampler within the environment wrappers to artificially bias the initial conditions toward these failure regimes, aiming to incentivize failure-state focused RL training.

4. Task IV: Residual Policy Finetuning

The Policy Decorator architecture was implemented to train a residual PPO agent alongside the frozen Diffusion base policy ($\pi_{final} = \pi_{base} + \alpha \cdot \pi_{res}$).

4.1. Challenges in Architectural Alignment

Bridging the two algorithms required overcoming what felt like a gauntlet of technical challenges to stabilize the PPO training during ManiSkill's GPU vectorization:

1. **Multimodal Key Mismatches:** The PPO baseline expected a single `rgb` key, while the Diffusion policy required separate `rgb` and `depth` buffers. It seemed necessary to engineer a `UnifiedRGBDWrapper` that fused a 4-channel `rgbd` tensor for the PPO feature extractor while preserving the original keys for the base policy.
2. **Data Type Conflicts:** The environment natively provided `uint8` unsigned character tensors, which immediately crashed the PyTorch CNN expecting `float32` weights. Automatic type-casting and normalization ($[0, 1]$) were embedded strictly within the wrapper stack.
3. **Dynamic Shape Calculation:** The standard PPO CNN hardcoded an output dimension of 1024, which triggered matrix multiplication errors when confronted with the flattened size of the 4-channel images and state vectors. The `NatureCNN` had to be surgically overridden to dynamically calculate matrix shapes via dummy forward passes initialized on the CPU.
4. **In-Place Tensor Mutations:** Because ManiSkill re-uses GPU memory buffers for efficiency, shifting the observation history array only duplicated stagnant memory references. The Diffusion agent effectively perceived a frozen robot and outputted paralyzed actions. This was ultimately resolved by implementing explicit deep tensor cloning (`.clone()`).

4.2. Finetuning Metrics and Convergence

Monitoring the live Tensorboard logs during the 1.5-hour residual finetuning process provided what felt like critical insights into the agent's learning dynamics. As depicted in Figure 4, the policy loss seemed to stabilize after the initial exploration phase. Notably, the training reward (Figure 5) exhibited an interesting pattern: the reward initially went down as the agent explored and deviated from the base policy, but it appeared to be steadily growing back up, perhaps as it discovered successful corrective actions.

Figure 4. Tensorboard logs illustrating the residual policy loss metrics stabilizing over time.

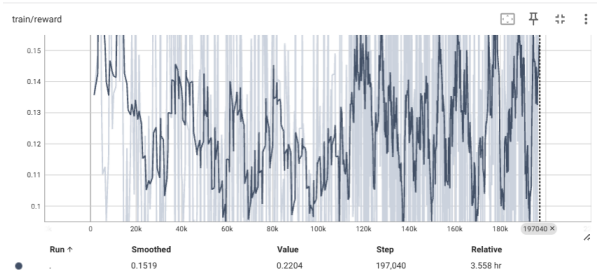

Figure 5. Tensorboard logs showing the training reward initially decreasing before appearing to steadily grow back up.

4.3. Catastrophic Failure and Alpha Bottleneck

Before fully correcting the LLM reward hacking and scalar bounds, the initial residual evaluations yielded exceptionally poor results:

- **Initial Nominal Performance:** 0.08% to 0.32% average success.
- **Initial Failure-Biased Performance:** 0.00% to 0.40% average success.

Challenges Faced: This was suspected to be due to what could be called the "Alpha Bottleneck." Because the baseline IL policy was incredibly weak (0.64% baseline success), a standard residual magnitude ($\alpha = 0.05$) lacked the mathematical authority to alter the trajectory sufficiently. The agent was effectively anchored to a failing base trajectory. To rectify this, it seemed best to increase the residual authority ($\alpha = 1.0$), and enforce the strictly negative LLM reward.

Unfortunately, despite these architectural fixes, the residual finetuning suffered catastrophic failure. The agent seemed to succumb to a "running away" local optimum to minimize negative reward accumulation. The best nominal success rate peaked at a mere **0.32%**, and on the failure-biased distribution, the success rate remained at **0.00%**.

Table 1. Final Agent Evaluation (250 Rollouts x 5 Seeds)

Distribution	Base Model	Residual Model
Nominal	0.64%	0.32%
Failure-Biased	0.30%	0.00%

5. Conclusion

While an architecture capable of supporting Failure-Aware Autonomous Continual RL was successfully designed, the practical execution unfortunately yielded catastrophic failure. Through careful visual diagnosis and metric tracking, severe reward hacking and reversed angle calculations were identified. The manual engineering required to continually patch `custom_wrappers.py` and `custom_reward.py` suggests that, despite LLM assistance, aligning residual RL with complex VLA models remains highly unstable and compute-intensive.

References

[1] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. *arXiv preprint arXiv:2310.12931*, 2023. 2, 3

[2] Wenhao Yu et al. Language to rewards for robotic skill synthesis. *Conference on Robot Learning (CoRL)*, 2023. 2

[3] Xiu Yuan, Tongzhou Mu, Stone Tao, Yunhao Fang, Mengke Zhang, and Hao Su. Policy decorator: Model-agnostic online refinement for large policy model. *arXiv preprint arXiv:2412.13630*, 2024. 3